ChatGPT

MediScribe AI: Architecture and Research Insights

Clinicians need a modern, integrated journaling app that combines rich note-taking (handwriting, voice, images) with intelligent AI tools. MediScribe AI envisions a **mobile (iOS/Android) and desktop** app where clinicians can journal patient encounters using typed or handwritten notes (transcribed via OCR). The backend uses **Retrieval-Augmented Generation (RAG)** to ground AI-generated summaries and prompts in the user's own notes and standardized medical knowledge (e.g. ontologies like the Omaha System). Below we survey the key technologies, competitor UIs, and research foundations shaping this design.

AI Models and APIs

We plan to support multiple LLM providers (OpenAI, Anthropic, Google). These platforms offer API endpoints and developer tools for building AI-enabled apps:

- **OpenAI (GPT-4 / GPT-5)**: The OpenAI API (same pricing as Playground usage) charges per token. For example, GPT-4 Turbo (e.g. model `gpt-4-0613`) costs roughly *$15.00 per 1M input tokens and $30.00 per 1M output tokens* [1] (≈$0.015 input / $0.03 output per 1K tokens). In contrast, GPT-3.5-Turbo is far cheaper (about *$0.25/$0.75 per 1M tokens* [1]). The API supports chat or completions endpoints, function/tool calls, and multimodal inputs. We would use the Playground **to prototype prompts quickly** before coding. OpenAI's docs emphasize RAG and tool calls (e.g. calculators, search, code execution) to improve accuracy [2] [3].

- **Anthropic Claude**: The Claude Developer Platform offers text APIs plus new agentic "Skills." Claude charges per token similar to OpenAI, and *separately charges for certain tools*. For example, Claude's web search tool costs **$10 per 1,000 searches** (on top of normal token costs) [4]. The WebFetch tool has no extra fee (only standard token usage for fetched content) [5]. Claude's API supports streaming, structured JSON outputs, citations, vision, etc. (The Claude docs feature extensive guides on prompting best practices and tool integration [6] [4].)

- **Google Gemini (via AI Studio)**: Google's Gemini models (formerly part of Vertex AI) can be accessed via an API key. Gemini offers RAG tools like a **"File Search"** (built-in vector store) which automates chunking and retrieval [7]. Pricing varies by model: e.g. *Gemini 3 Flash (preview)* is about **$0.50 per 1M input tokens and $3.00 per 1M output tokens** [8]. (By comparison, Gemini 2.5 Pro is $10 per 1M output tokens [9].) Google's docs describe uploading files to a vector store and querying it to get grounded answers with citations [7] [10]. We plan to use Gemini File Search for RAG over any uploaded PDFs (e.g. medical guidelines) and patient records.

Each platform has SDKs and REST APIs. OpenAI's "Build your app" guides and Anthropic's docs highlight best practices (prompt engineering, evals, tool use) [11] [12]. We will prototype prompts in the Playground/Console before coding. All models require internet and authentication, and we must manage tokens per request (token limits up to 128K or more for some models). Model choice can be dynamic (e.g. default to cheaper GPT-3.5-Turbo for simple chat, switch to GPT-4 for complex tasks).

Data Storage & Vector DB

We need a backend to store user data, app state, and support RAG retrieval:

- **Firebase (Firestore)**: Cloud Firestore in Firebase will hold user profiles, app settings, and structured data (journals metadata, tags). Firestore's **free tier** includes 1 GiB storage, 50k document reads/day, 20k writes/day, 20k deletes/day [13]. After that, billing applies (typically ~$0.18 per 100k reads, ~$0.18 per 100k writes). Firestore auto-scales with concurrency and offers offline sync for mobile. Real-time Database or Firestore can sync live updates (e.g. shared notes). We will also use **Firebase Authentication** (email/SMS) for user login, and **Firebase Storage** or Cloud Storage for user-uploaded files (audio recordings, PDFs).

- **Pinecone (Vector Database)**: To implement RAG over user content (and perhaps medical literature), we store text embeddings in Pinecone. On the free "Starter" plan, Pinecone allows ~100,000 vectors (at 1536 dimensions) in a single index [14]. Rate limits are 100 queries/sec and 100 upserts/sec per namespace [15]. The free plan quotas include 1,000,000 read units/month and 2,000,000 write units/month [16]. This should handle a modest user base in prototype. Each patient note (or chunk) is embedded (using, e.g. OpenAI or Gemini embeddings) and upserted to Pinecone. At query time, we retrieve top-k similar chunks. (Pinecone's SDKs make upserts and queries simple.) If we scale, we could upgrade to a paid plan or use Pinecone's dedicated pods for higher throughput.

- **OCR and Handwriting (MyScript + Vision API)**: For handwritten notes or images, we use a combination of OCR. Google Cloud Vision or Apple VisionKit can extract printed or handwritten text. For rich digital ink, **MyScript's iInk SDK** provides advanced handwriting recognition. Notably, MyScript's "Text Document" component supports **unlimited document length** (no fixed-page size) [17], letting users write arbitrarily long notes by hand. MyScript also recognizes math, diagrams, and shapes. We would integrate MyScript's SDK (or call their cloud API) in the app for on-device handwriting conversion. This aligns with use cases like in Moleskine Smart notebooks [18] [19].

- **Firebase's AI Studio Integration**: Firebase offers an "AI Studio" with GenAI tools and a Firestore extension to index content for RAG. We can leverage Firebase's upcoming RAG support (e.g. Gemini integration and Firestore search) as part of prototyping to reduce dev work, using Firestore documents as vector sources.

Frontend UI / UX Comparison

The user requested a **clean, intuitive UI** exceeding competitors (Journal app, Obsidian, Scrivener):

- **Journal (Apple's iOS Journal)**: Journal presents entries as **cards on a scrolling feed**, each card showing date, text, and embedded items. Tapping "+" opens a **full-screen prompt** with *smart suggestions*: it auto-groups your photos, workouts, locations, music, etc. into prompts to kickstart an entry [20]. The UI is minimalist and polished (blur effects, translucent backgrounds) [21]. Formatting is limited (bold/italic/etc), and content is laid out in a compact, grid-like view per entry [22]. We will draw from this approach by using a clean card-based timeline and providing AI-assisted entry prompts (e.g. "Today's vitals", "lab results"...).

- **Obsidian**: Obsidian is a Markdown knowledge base. Its UI is pane-based (editor, preview, graph). Key features: **bi-directional linking** ("[[links]]") so users can interconnect notes (creating a

personal wiki) [23]. It also has a **graph view** to visualize note relationships [24]. Obsidian's plugin ecosystem (calendar, kanban, tasks) offers customizable UI. We want a journaling app to be more structured, but can borrow the idea of backlinking (e.g. linking patients, problems, medications) and an overview dashboard (e.g. calendar or graph of entries).

- **Scrivener**: Scrivener's interface is optimized for writers: a left-side **Binder** (tree of folders/files) organizes documents [25], and it has Outline and Corkboard views. Users can drag/drop scenes to reorder their outline [26]. For MediScribe, we can emulate a simplified Binder: e.g. a sidebar of patients or cases, each containing dated entries and sections (History, Plan, etc.). However, for mobile, we will likely use a more linear UI, perhaps with collapsible sections.

In summary, our **UI goals**: - Card-based journal timeline (like Apple Journal) with rich media. - Support handwriting/typing, with intuitive gestures (pinch to zoom text, scribble-to-text). - Seamless linking/tagging of clinical concepts (inspired by Obsidian). - Multi-pane editor on desktop (like Scrivener), and simple single-pane on mobile. - Themes and customization to ensure it "looks great" (Obsidian mentions themes, Journal emphasizes design).

RAG and Clinical NLP

Retrieval-Augmented Generation (RAG) is central to MediScribe's intelligence. In RAG, an LLM is **augmented with an external retrieval step**: user queries (or notes) trigger a search of a knowledge base; the LLM then generates answers grounded in the retrieved context [27] [28]. A useful analogy: like combining GPT-4's "knowledge" with real-time reference lookups. Key aspects:

- **RAG Process**: As explained by K2View, RAG injects fresh, trusted data into the LLM prompt to improve accuracy [29] [30]. The typical flow (illustrated below [31]) is: (1) user prompt → (2) retrieval model queries databases/docs → (3) retrieved documents are appended to the prompt → (4) LLM generates the response. Embeddings enable fast similarity search over a vector index (Pinecone).

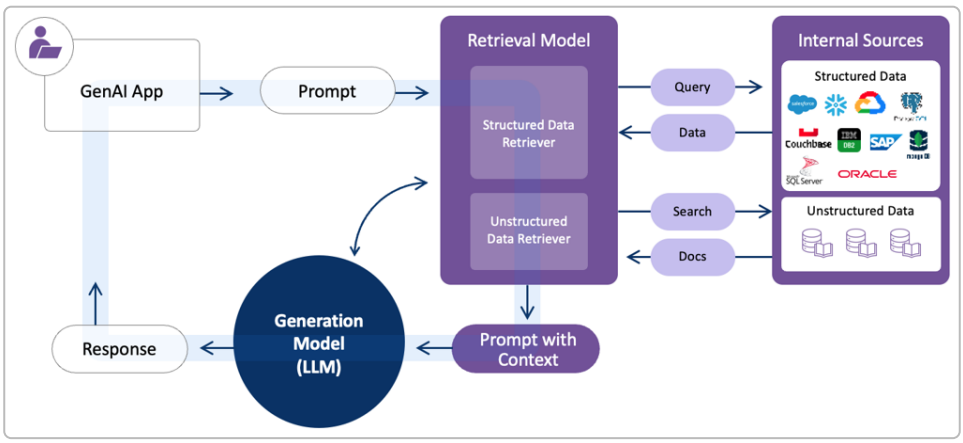


Figure: RAG architecture – the GenAI app sends a prompt to the Retrieval Model, which fetches relevant data (structured from systems or unstructured docs) and crafts an augmented prompt for the LLM; the LLM then produces a grounded answer [31]. Using RAG significantly **reduces hallucinations** by grounding answers in real data [32].

- **Clinical RAG/NLP**: Applying RAG in healthcare, recent studies show LLMs can match expert rule-based extraction when aided by RAG [33]. For example, a 2025 study used an LLM+RAG pipeline to map nurse-patient dialog to standardized Omaha System terms [33] [34]. They embedded both

patient utterances and clinical ontology entries into Pinecone, retrieved relevant medical terms, and had the LLM select the correct category. This boosted accuracy and consistency in problem identification. We will adopt a similar strategy: encode encounters and ontology strings with an embedding model, use Pinecone for retrieval, then let the LLM classify or summarize. The review [45] emphasizes RAG's promise for **knowledge-intensive tasks** and lists common RAG architectures.

- **OCR/Handwriting**: For scanned or handwritten notes, OCR bridges images to text [35]. Modern OCR (and AI) systems now achieve high accuracy across fonts, languages and layouts [35]. We will run page scans through OCR before indexing in RAG. MyScript's neural-net recognition (trained on millions of samples) can convert handwritten paragraphs, math, or diagrams into structured text [17]. This makes scribbles "searchable" and integrable into the database for future retrieval.

- **Semantic Search**: We will use medical embeddings (e.g. Bio+Clinical BERT or an OpenAI clinical embed) to capture meaning. Whenever a clinician types or draws a phrase (e.g. "shortness of breath"), we query Pinecone for similar existing notes or standards (e.g. "abnormal breathing pattern") [34]. The retrieved pieces are fed into GPT/Claude/Gemini to produce a summary or suggestion. Citations from the source text can be shown (just as Gemini's File Search returns "grounding metadata" for UI display [36]).

Overall, the NLP pipeline is: - Transcribe speech/handwriting to text (Speech→text via STT, MyScript for ink, OCR for images). - Chunk and embed text (using an embeddings model), upsert into Pinecone. - Upon query or generation, retrieve top-k relevant chunks, and feed them (plus prompt) into the LLM. - LLM generates output (notes, summaries, questions, etc.) with references.

Architecture & Cost Estimates

Prototype Architecture: (See Figure above) Users interact via a mobile or web UI. Content flows to a Firebase backend (Firestore + Storage) and to our vector store (Pinecone). LLM queries go to the chosen API (OpenAI, Claude, or Gemini). We will implement a microservice that, given a user prompt, (a) extracts any text/images, (b) queries Pinecone for context, (c) calls the LLM, and (d) returns a structured answer. Components:

- **Frontend**: React Native or Flutter for cross-platform mobile; React for web. Obsidian-like rich text editors (e.g. SlateJS) for note-taking. UI frameworks like Flutter or SwiftUI can mimic the sleek card designs of the Journal app.
- **API Layer**: A Node/Python backend (e.g. Firebase Cloud Functions or a separate server on Google Cloud Run) that communicates with LLM APIs and Pinecone. We will use the official client SDKs (OpenAI Python, Pinecone Python, Google GenAI SDK).
- **Database**: Firestore for user/journal metadata (fast real-time sync). Pinecone for semantic search. (If needed later, a relational DB on Cloud SQL for reporting.)
- **Storage**: Firebase Storage (or Cloud Storage) for raw files (scans, audio). These can be sent to OCR or speech-to-text models via cloud services.
- **Authentication/Security**: Firebase Auth (with multi-factor if needed). Data encryption at rest (Firestore and Pinecone support encryption). HIPAA compliance: Firestore/GCP can be HIPAA-ready if on a covered project; we will ensure business associate agreements.
- **Costs**:
- **LLM usage**: Using GPT-4 Turbo extensively could cost ~$0.03 per 1K output tokens [1]. Claude's web search adds $10/1K searches [4] (we'll use it judiciously). If the app's dialogue is ~100 tokens per query, that's ~$0.003 per query for GPT-4; GPT-3.5 is even cheaper.

- **Gemini**: For prototyping, the free tier (200K tokens) may suffice. Production costs: e.g. Gemini 3 Flash at $3 per 1M output tokens [8] (~$0.003 per 1K tokens), similar to GPT-4.
- **Firebase**: Small usage under free tier (50K reads/day, 20K writes/day) is free [13]. Beyond that, Firestore costs about $0.06/100K reads and $0.18/100K writes. Auth (sign-in) is free. Storage is ~$0.026/GB-month.
- **Pinecone**: Starter plan is free up to 1M reads/mo, 2M writes/mo [16], with ~100K vectors [14]. Paid plans start around $0.096/hour for a single pod (for higher scale) [37].
- **MyScript**: Development is free (25-device license). Production licensing is per-device or per-seat (contact MyScript for pricing). No per-call fees listed.
- **Compute**: GCP/Cloud Run can host our services; assume $10–$20/month for light usage. (The heavy cost driver will be LLM tokens.)

By mixing cheaper models (GPT-3.5 for routine chat, GPT-4 for complex tasks), and caching RAG results (reuse retrieved chunks), we can keep costs moderate. If needed, Gemini's built-in File Search and Google's context caching ($0.05/1M tokens) could save on embedding calls [38].

Summary: MediScribe's architecture leverages modern AI: it uses **HTTP APIs for LLMs** (with cost ~ cents per query [1] [8]), a **cloud NoSQL DB (Firestore)** with generous free quotas [13], and a **vector DB (Pinecone)** for RAG (free up to ~100k vectors [14]). Frontend UX combines the best of Journaling (clean cards) and note-linking (like Obsidian/Scrivener). This design is informed by recent research in RAG and clinical NLP [27] [33], ensuring clinicians can both record rich notes and get AI-powered insights grounded in real patient data.

Sources: We drew on official docs and literature: OpenAI and Claude pricing info [1] [4]; Google Gemini API guides [8] [7]; Firebase limits [13]; Pinecone starter plan details [16] [14]; MyScript SDK docs [17]; UI reviews for Journal, Obsidian, Scrivener [20] [23] [25]; and academic surveys on RAG, OCR, clinical NLP [27] [35] [33]. These ensure our plan is current and grounded in best practices.

---

[1] Pricing | OpenAI API
https://developers.openai.com/api/docs/pricing/

[2] [11] AI app development: Concept to production
https://developers.openai.com/tracks/ai-application-development/

[3] [28] [29] [30] [31] [32] What is Retrieval-Augmented Generation (RAG)? A Practical Guide
https://www.k2view.com/what-is-retrieval-augmented-generation

[4] [5] Pricing - Claude API Docs
https://platform.claude.com/docs/en/about-claude/pricing

[6] [12] Get started with Claude - Claude API Docs
https://platform.claude.com/docs/en/get-started

[7] [10] [36] Gemini API's New "File Search" Tool — Built-In RAG for Everyone | by Abdul Qadir | Medium
https://medium.com/@abdulkadir9929/gemini-apis-new-file-search-tool-built-in-rag-for-everyone-e990c054dcff

[8] [9] [38] Gemini Developer API pricing | Gemini API | Google AI for Developers
https://ai.google.dev/gemini-api/docs/pricing

[13] Usage and limits | Firestore | Firebase
https://firebase.google.com/docs/firestore/quotas

[14] Limit of vectors on free plan - Support - Pinecone Community
https://community.pinecone.io/t/limit-of-vectors-on-free-plan/3821

15 16 Pinecone Database limits - Pinecone Docs
https://docs.pinecone.io/reference/api/database-limits

17 18 19 Handwriting recognition & digital ink SDK | MyScript
https://www.myscript.com/sdk/

20 21 22 Apple's Journal App: Journaling for All? - MacStories
https://www.macstories.net/reviews/apples-journal-app-journaling-for-all/

23 24 Obsidian - Sharpen your thinking
https://obsidian.md/

25 26 Outlining with the Scrivener Binder - Literature & Latte
https://www.literatureandlatte.com/blog/outlining-with-the-scrivener-binder

27 [2508.06401] A Systematic Literature Review of Retrieval-Augmented Generation: Techniques, Metrics, and Challenges
https://arxiv.org/abs/2508.06401

33 34 From Conversation to Standardized Terminology: An LLM-RAG Approach for Automated Health Problem Identification in Home Healthcare - PMC
https://pmc.ncbi.nlm.nih.gov/articles/PMC12849705/

35 (PDF) The Evolution and Impact of Optical Character Recognition (OCR) in Digital Transformation
https://www.researchgate.net/publication/396482259_The_Evolution_and_Impact_of_Optical_Character_Recognition_OCR_in_Digital_Transformation

37 A Guide to Pinecone Pricing - Tiger Data
https://www.tigerdata.com/blog/a-guide-to-pinecone-pricing